Lívia Caroline Rodrigues Pereira - 20240240003

ÁRVORES DE CLASSIFICAÇÃO 2D

Entrega do código

```
Melhor Split Encontrado: Atributo x1 <= 0.6228
Ganho de Informação: 0.1195

Tabela de Contagens por Classe nos Nós Filhos:
```

	Nó Esquerdo (<= t)	Nó Direito (> t)
Classe (y)		
0	248	59
1	59	134

Contagens por classe do melhor split inicial

```
Tamanho do Treino: 351 amostras
Tamanho do Teste: 149 amostras

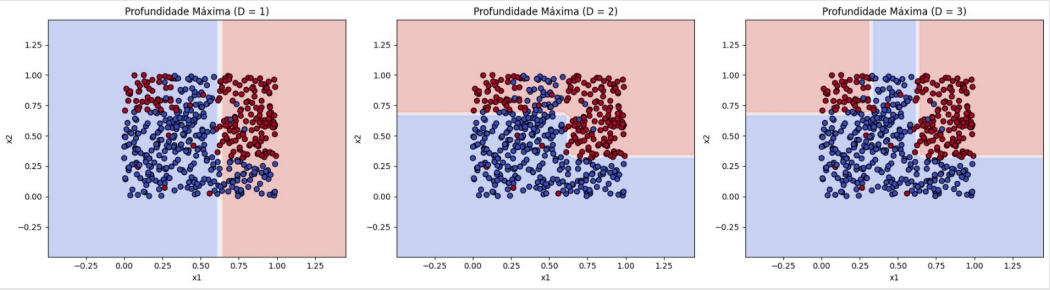
--- Resultados da Avaliação no Teste ---
Acurácia: 0.9664

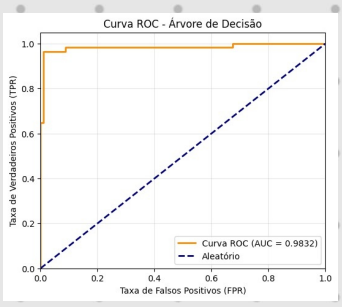
Matriz de Confusão:
```

Classe Prevista	0	1
Classe Real (y)		
0	91	1
1	4	53

Resultados do treino/teste

Fronteira de decisão





Esboço explicativo

Dados

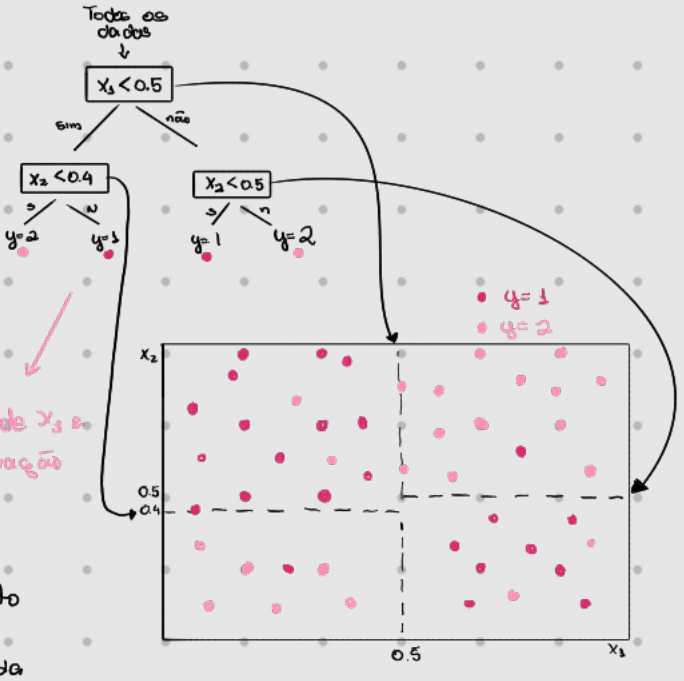
X_1	X_2	y
0.1	0.4	1
0.2	0.3	2
⋮	⋮	⋮
0.7	0.7	2
0.6	0.4	1

Split de dados (X) para tentar prever o y

Tento as medias dos X_1 e X_2 como candidatos e pego aquela que maximizar o Ganho de Informação

Médias 2 a 2:

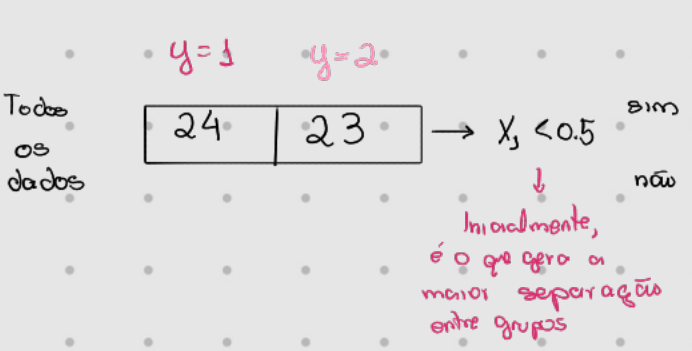
X_1: $\frac{0.1+0.2}{2}$, ..., $\frac{0.7+0.6}{2}$

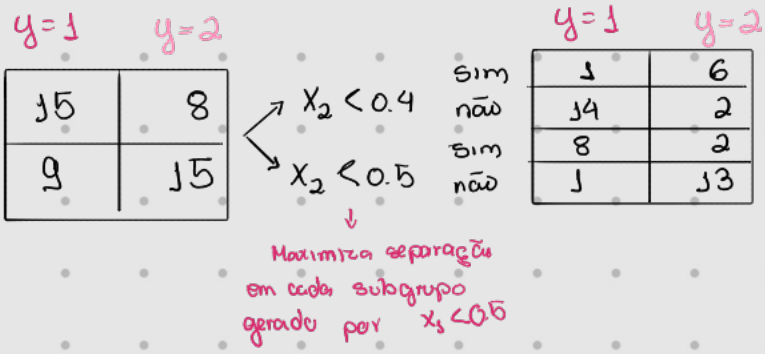


Testei para as variações de X_1 e X_2 possíveis (splits) e escolhi a configuração que maximiza o ganho de informação.

↳ Com todos os dados, todos os splits (tanto para X_1 quanto para X_2) são testados. $X_1 < 0.5$ maximizou o ganho de informação. Agora, em cada ramo, ele testa de novo todos os splits (X_1 e X_2) restantes e escolhe os que maximizam!

O passo a passo da divisão ficou assim:





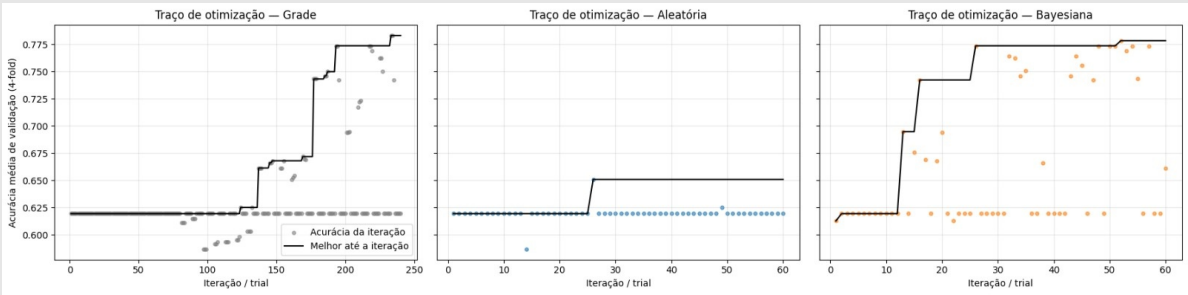
TUNING DE HIPERPARÂMETROS

Hiperparâmetro	Grade (subconjunto discreto)	Aleatória	Bayesiana (Optuna/TPE)
D	{1, 2, 3, 4, 5, 6}	uniforme em {1,...,6}	`suggest_int(1, 6)`
n_{min}	{5, 10, 20, 30, 50}	uniforme em {5, 10, 20, 30, 50}	`suggest_categorical`
`min_gain`	{0, 0.001, 0.0025, 0.005, 0.01, 0.02, 0.035, 0.05}	contínuo $\mathcal{U}[0, 0.05]$	`suggest_float(0, 0.05)`

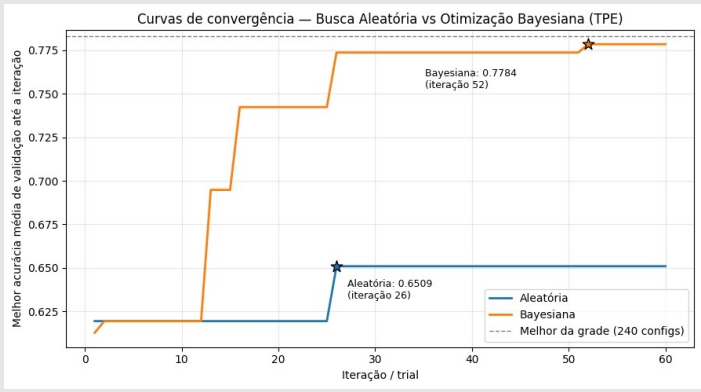
Espaço de busca

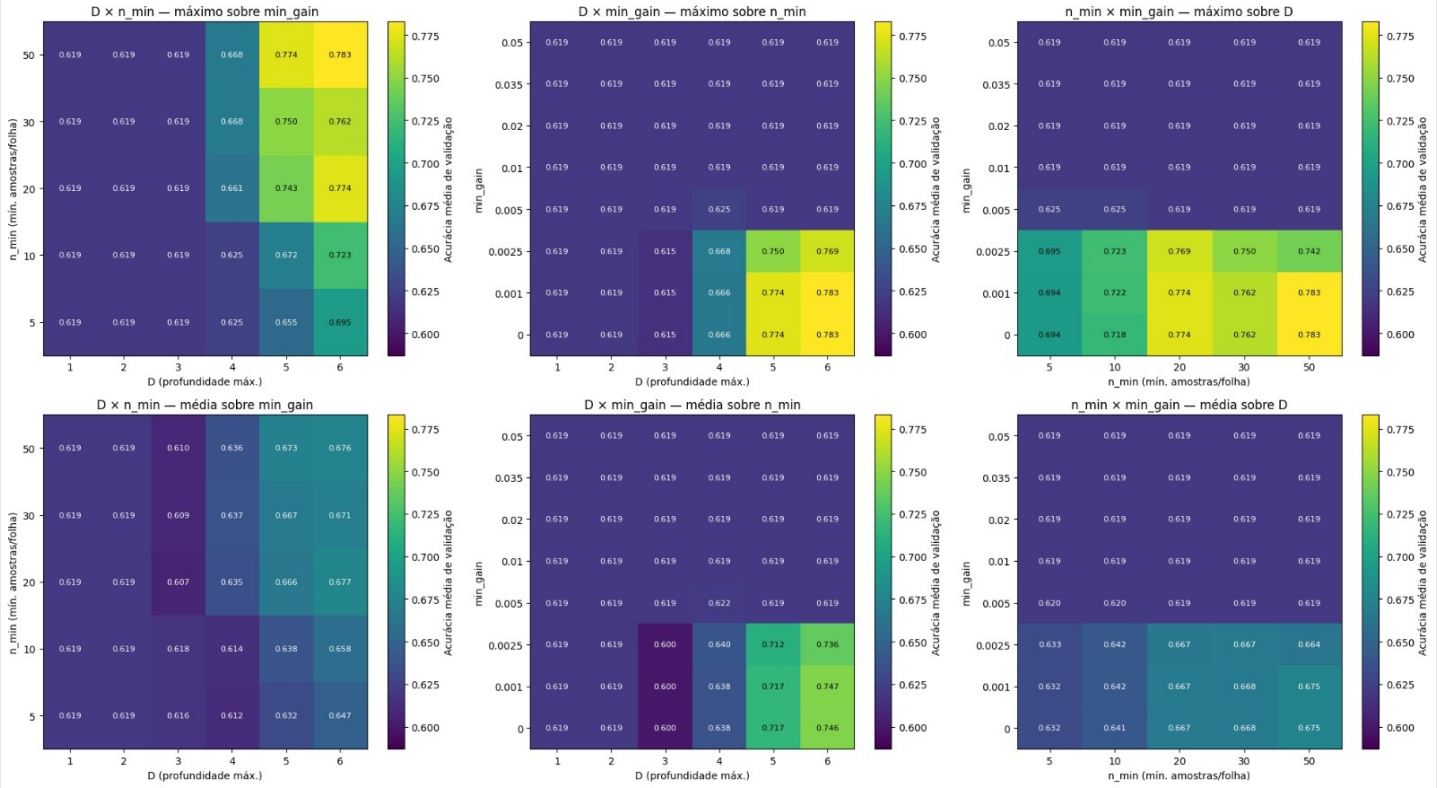
Estratégia	D*	n_min*	min_gain*	Acc. média CV	Desvio entre folds	Iteração do ótimo	Configurações	Treinamentos (config × K)	Tempo (s)
Grade	6	50	0.00000	0.7832	0.1045	233	240	960	9.01
Aleatória	5	5	0.00037	0.6509	0.0855	26	60	240	1.79
Bayesiana	6	20	0.00199	0.7784	0.0969	52	60	240	2.81

Resultado das 3 estratégias



Graficos

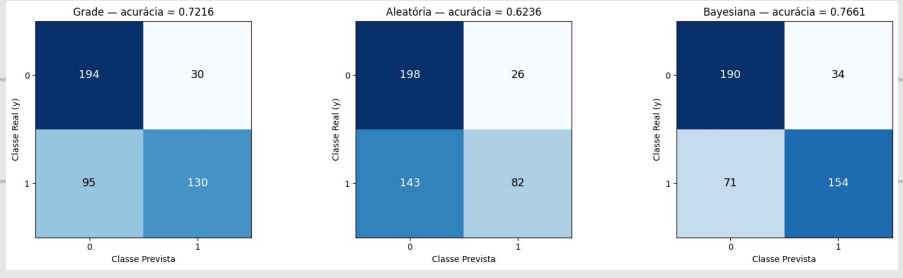




As curvas de convergência mostram uma diferença clara entre as estratégias: Enquanto a otimização Bayesiana melhora muito sua acurácia de validação (0.62 → 0.77) em relativamente poucas iterações, a busca aleatória fica "presa" em um patamar e, ao fim, tem uma acurácia menor. Isso pode ser explicado pelo comportamento do hiperparâmetro min-gain, em que apenas valores abaixo de 0.003 produzem árvores realmente úteis; como esses valores estão em apenas 5% do intervalo, a busca aleatória quase não os escolhe, enquanto TPE aprende que a faixa desses valores de min-gain é onde deve atacar.

Isso é confirmado pelos mapas de calor, em que as acurácias altas se concentram em uma região pequena, onde min_gain < 0.003 ou $D \in \{5,6\}$, o que coloca esses dois hiperparâmetros como aqueles de maior impacto. Quanto à estabilidade, árvores rasas e configurações com min-gain alto são muito estáveis, mas em patamar ruim; assim, os melhores modelos são também os mais sensíveis ao treino. Por fim, os hiperparâmetros mostram interações claras, tal que: aumentar D só ajuda quando min-gain é pequeno ou n_min grande (árvores profundas precisam de folhas grandes). De modo geral, o melhor modelo vem de árvore profunda regularizada por n_min.

Avaliação no teste



A árvore escolhida pela otimização Bayesiana obteve melhor acurácia. Chama atenção a inversão entre grade e Bayesiana, tal que a grade tinha maior acurácia na validação mas ficou atrás no teste, o que é coerente com o alto desvio entre folds. Ao escolher a melhor entre configurações quase empatadas, a Grade tende a favorecer quem "teve sorte" no treino, superestimando seu desempenho. As matrizes de confusão ainda mostram que nas três estratégias a maioria dos erros é de falsos negativos,

sugerindo que as árvores ainda não têm profundidade suficiente para recortar todas as regiões da classe 1. Por fim, a acurácia medida com y_true é praticamente igual à medida com y (0.78 contra 0.766 na Bayesiana), e a discordância entre y e y_true é de apenas 3.3%, mostrando que o ruído do rótulo não é limitante e a perda de desempenho vem do próprio modelo

Parágrafo técnico

Em custo computacional, a busca por grade foi a mais cara: avaliou 240 configurações (960 treinamentos e 96) contra 60 configurações da busca aleatória e Bayesiana. Na Bayesiana, parte do tempo vem do próprio TPE, que precisa ajustar um modelo sobre as iterações anteriores antes de sugerir a próxima. Apesar do alto custo, Grade não apresenta a maior eficiência amostral, mas sim Bayesiana, que concentra as sugestões de incremento do próximo treino exatamente onde os hiperparâmetros são mais eficientes, enquanto grade faz busca exaustiva e pode não chegar onde quer (pois as combinações são pré-definidas). As melhores configurações apresentam desvio alto, ou pouca estabilidade, explicando porque a vantagem da grade não se mantem no teste. Por fim, os hiperparâmetros ótimos surgiram nas bordas do espaço, o que somado ao fato da acurácia (0.78) estar muito abaixo do teto imposto pelo ruído (0.99), indica que a grade estava estreita e valeria ampliar D e n_min e refinar min_gain perto de zero.